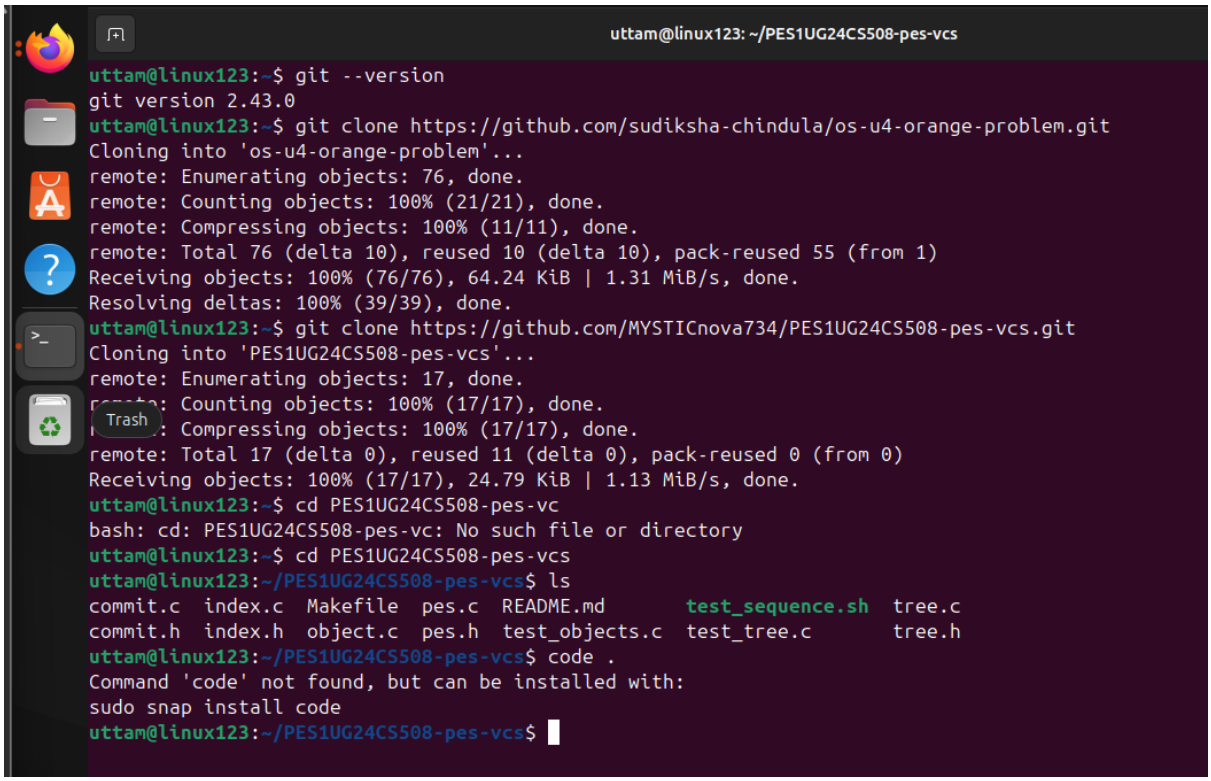
OPERTATING SYSTEM

ORANGE – PROBLEM

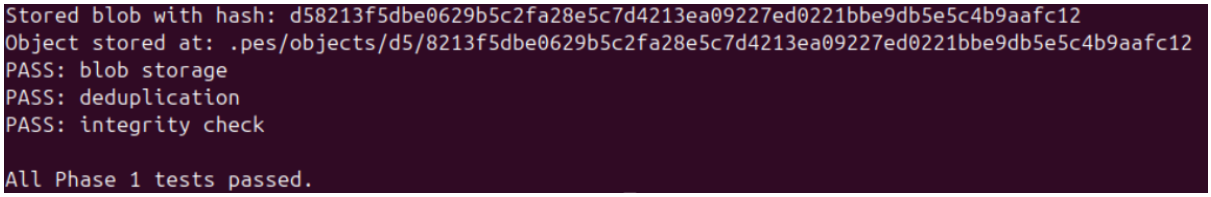
UTTAM B S

PES1UG24CS508



Phase 1: Object Storage

Screenshot 1A: test_objects



Screenshot 1B: Object Store Structure

```
.pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
.pes/objects/25/ef1fa07ea68a52f800dc80756ee6b7ae34b337afedb9b46a1af8e11ec4f476
.pes/objects/2a/594d39232787fba8eb7287418aec99c8fc2ecdaf5aaf2e650eda471e566fcf
```

Phase 2: Tree Objects

Screenshot 2A: test_tree Output

```
gcc -Wall -Wextra -O2 -c test_tree.c -o test_tree.o
gcc -o test_tree test_tree.o object.o tree.o -lcrypto
```

```
Serialized tree: 139 bytes
PASS: tree serialize/parse roundtrip
PASS: tree deterministic serialization

All Phase 2 tests passed.
```

Screenshot 2B: Raw Tree Object

```
a6e6719f35f9f7e65d6f606199f7f | head -20
00000000: 626c 6f62 2036 0077 6f72 6c64 0a         blob 6.world.
```

Phase 3: Index (Staging Area)

Screenshot 3A:

```
Staged changes:
  staged:     file1.txt
  staged:     file2.txt

Unstaged changes:
  (nothing to show)

Untracked files:
  untracked:  tree.c
  untracked:  test_sequence.sh
  untracked:  pes.c
  untracked:  commit.c
  untracked:  test_objects.c
  untracked:  test_tree
  untracked:  index.c
  untracked:  test_tree.c
  untracked:  index.h
  untracked:  tree.h
  untracked:  README.md
  untracked:  commit.h
  untracked:  object.c
  untracked:  Makefile
  untracked:  pes.h
  untracked:  test_objects
```

Screenshot 3B: Index File Contents

```
100664 2cf8d83d9ee29543b34a87727421fdecb7e3f3a183d337639025de576db9ebb4 1776424638 6 file1.txt
100664 e00c50e16a2df38f8d6bf809e181ad0248da6e6719f35f9f7e65d6f606199f7f 1776424638 6 file2.txt
```

Phase 4: Commits

Screenshot 4A: Commit Log

```
commit 5dfb36ccda68e3cc58a8660d3b763ae1a9a6234b33ad44c70295ce93c46f7589
Author: PES User <pes@localhost>
Date:   1776426791

    Add bye file

commit a522ca894927e0196fb042291093a880e2d8f47708fd156a67ae240698157ffe
Author: PES User <pes@localhost>
Date:   1776426763

    Add worldile

commit e9b16c9c971f1492ebf4f0e94e6987e85d2d6dfc3f744312e55a72fbf3cb196d
Author: PES User <pes@localhost>
Date:   1776426686

    Initial committial commiQ
```

Screenshot 4B: Object Growth

```
.pes/HEAD
.pes/index
.pes/objects/05/c8d777d0be85748b6788d1b5caf7d69754e6e68060c265ca21cbd6c05e5f43
.pes/objects/2c/f8d83d9ee29543b34a87727421fdecb7e3f3a183d337639025de576db9ebb4
.pes/objects/48/1407f7237f7bd749dca31e2d8422bafe211741d5556f007e0faf821ffc7179
.pes/objects/5d/fb36ccda68e3cc58a8660d3b763ae1a9a6234b33ad44c70295ce93c46f7589
.pes/objects/66/224663d23e6f4d9de9e2c7e6d8764305a92a3830a1a52d3d5f4aa8007b5c39
.pes/objects/7d/c3b9d4cee22f645e501f8aaa822f6777b3198434adc74491632478799e9e8b
.pes/objects/92/19086282c09f0db9088484e1ef89f1b05ecfb420a8784a6de327022771fbec
.pes/objects/a5/22ca894927e0196fb042291093a880e2d8f47708fd156a67ae240698157ffe
.pes/objects/e0/0c50e16a2df38f8d6bf809e181ad0248da6e6719f35f9f7e65d6f606199f7f
.pes/objects/e9/b16c9c971f1492ebf4f0e94e6987e85d2d6dfc3f744312e55a72fbf3cb196d
.pes/objects/fe/bed41618fe89b49ea9b82684254a1e74aa7f80fdba38fc84b592ccca955267
.pes/refs/heads/main
```

Screenshot 4C: HEAD and Branch

```
5dfb36ccda68e3cc58a8660d3b763ae1a9a6234b33ad44c70295ce93c46f7589
```

```
ref: refs/heads/main
```

Analysis Questions

Q5.1: Implementing pes checkout <branch>

A branch in PES is a file located in .pes/refs/heads/ that stores a commit hash. Creating a branch is equivalent to creating this file.

To implement pes checkout <branch>:

Steps:

1. Check if .pes/refs/heads/<branch> exists.
2. Update .pes/HEAD to:

ref: refs/heads/<branch>

3. Read the commit hash from the branch file.
4. Load the corresponding commit and retrieve its tree.

Update the Working Directory:

- Add or remove files to match the commit
- Restore file contents from blob objects
- Recreate directory structure if necessary

Update Index:

- Update .pes/index to reflect the checked-out state

Complexity:

This operation is complex because it must:

- Handle uncommitted changes safely
- Prevent accidental overwrites
- Manage file deletions correctly
- Maintain consistency between HEAD, index, and working directory

---

Q5.2: Detecting a Dirty Working Directory Conflict

To detect conflicts when switching branches:

Step 1: Check for Modified Files

For each tracked file in the index:

- Use stat() to check file size and modification time
- If it differs from the index entry → file is modified (dirty)

Step 2: Compare with Target Branch

- Load target branch's commit and tree
- For each file, compare:
    - Index hash vs target tree hash

Conflict Condition:

A conflict occurs if:

- File is modified in working directory (≠ index), **AND**
- File differs in target branch (index hash ≠ target hash)

Result:

Checkout must refuse to proceed to prevent data loss.

Q5.3: Detached HEAD State

A detached HEAD occurs when .pes/HEAD contains a commit hash instead of a branch reference.

What Happens:

- Commits can still be created
- These commits are not attached to any branch
- They may become unreachable when switching branches

Recovery Methods:

- Create a new branch:

pes branch <new-branch>

- Or manually point a branch to the commit hash

This ensures commits are preserved and accessible.

Garbage Collection and Space Reclamation

Q6.1: Deleting Unreachable Objects

Algorithm: Mark-and-Sweep

Step 1: Mark Reachable Objects

- Start from all branch heads in .pes/refs/heads/
- Add commit hashes to a set
- Recursively traverse:
    - Commit → parent commits
    - Commit → tree
    - Tree → blobs and subtrees
- Continue until no new objects are found

Step 2: Sweep Unreachable Objects

- Scan .pes/objects/
- Delete objects not present in reachable set

Data Structure:

- Use a **Hash Set**
    - O(1) lookup
    - Avoid duplicate visits

Estimation:

For:

- 100,000 commits
- 50 branches

Approximate objects:

- Commits: ~100,000
- Trees: ~100,000
- Blobs: ~500,000 (assuming ~5 per commit)

Total ≈ 700,000 objects

Q6.2: GC and Race Conditions

Problem:

Running garbage collection during a commit can cause corruption.

Race Condition Scenario:

1. Commit creates blobs and trees
2. These objects are not yet referenced by any branch
3. GC runs and marks them as unreachable
4. GC deletes them
5. Commit finishes and references missing objects → corruption

How Git Avoids This:

- Uses **atomic updates** (temporary files + rename)
- Introduces a **grace period** for new objects
- Uses **locks and reflogs** to keep objects reachable temporarily